C60 — PCB Connector Alignment Detection

Project Report

Prepared by: Soham Bhattacharyya

1. Project Overview

This is a computer-vision based automated optical inspection (AOI) project focused on verifying the presence and alignment of a PCB connector on an assembly line. The system uses a YOLO object detection model to identify whether a connector is correctly present in an image frame, flagging misaligned or missing connectors as part of a quality-control checkpoint. The trained model is intended to run inference on-device (edge deployment) as part of an automated inspection station.

1.1 In Scope

- Data collection of PCB connector images under varying lighting, angle, and placement conditions representative of the production line.
- Annotation of a single object class ("connector") using bounding boxes.
- Data augmentation strategy tailored to the connector's physical constraints (no flips, since the connector is polarized/asymmetric; small-angle rotation, brightness/contrast/saturation jitter, mild noise and blur).
- Fine-tuning a pretrained YOLOv11n model for single-class object detection.
- Iterative training, validation, and evaluation using precision, recall, and mAP metrics.
- Packaging of trained weights and evaluation artifacts for deployment on edge hardware (Raspberry Pi 5).
- Driving 2 Raspberry Pi Cam through a Camera Multiplexer.

1.2 Out of Scope

- Multi-class defect classification (e.g. distinguishing bent pins, solder defects, or component identity) — this project detects presence/alignment of a single connector class only.
- Pose estimation or exact rotation-angle regression of the connector; the current model outputs an axis-aligned bounding box only, not an oriented box or keypoints.
- Full PCB inspection (other components, traces, solder joints) — scope is limited to the connector region.
- Real-time video pipeline integration, PLC/line-controller interfacing, and mechanical reject/sorting mechanisms — these are downstream integration concerns handled outside this repository.
- Data collection under conditions not represented in the current dataset (e.g. entirely different connector models/series, extreme occlusion, or non-fixture-mounted orientations).
- Model compression/quantization work beyond what is needed for basic edge deployment (formal INT8 quantization, pruning, etc are not covered in this phase).

2. Project Phases

Phase 1 — Data Collection

Capture of PCB connector images from the production/fixture setup, covering realistic variation in lighting, camera distance, and connector placement.

Phase 2 — Annotation & Dataset Preparation

Labeling connector bounding boxes in Roboflow, organizing images into train/validation/test splits, and exporting the dataset in a YOLO-compatible format.

Phase 3 — Data Augmentation Strategy

Defining an augmentation policy appropriate for a rigid, polarized connector: geometry-preserving transforms (small rotation, scale/crop jitter) and photometric transforms (brightness, contrast, saturation, noise), while explicitly excluding flips and large arbitrary rotations.

Phase 4 — Model Training

Fine-tuning a COCO-pretrained YOLOv11n model on the connector dataset, tracking loss and validation metrics across epochs, using early stopping and periodic checkpointing.

Phase 5 — Evaluation

Assessing trained checkpoints on held-out validation and test splits using precision, recall, mAP@0.5, and mAP@0.5:0.95, along with qualitative review of prediction samples and the confusion matrix.

Phase 6 — Deployment

Packaging the best-performing weights and supporting artifacts, and deploying the model to edge inference hardware (e.g. Raspberry Pi) integrated into the inspection station.

3. Progress & Training Iterations

Training and evaluation are being run on Kaggle notebooks (GPU-accelerated session), with the dataset pulled directly from Roboflow via its API at the start of each session. Each iteration below documents the environment/data setup, the training run, and the resulting metrics.

3.1 Training Iteration 1

Environment

- Platform: Kaggle Notebook, GPU accelerator (Tesla T4, 14.9 GB VRAM)
- Framework: Ultralytics YOLO (yolo11n.pt pretrained checkpoint, COCO weights)
- Dataset source: Roboflow project "bluarmor-od-wd3wq", version 1, exported in YOLOv11 format

Dataset Summary

Split	Images	Background (no connector)	With Connector (boxes)
Train	2220	1254	966
Validation	217	186	31
Test	111	88	23
Total	**2548**	**1528**	**1020**

The dataset contains a single class, "connector". Note the class imbalance: roughly 60% of images across all splits are background/pass frames with no annotated connector box, while the remainder contain one annotated connector instance. This is expected for an AOI-style presence/alignment check, but is worth monitoring in later iterations.

Code Cells

Cell 1: Environment Setup & Dataset Download

Installs the ultralytics and roboflow packages, verifies GPU availability (raises an error if no CUDA device is found), and authenticates with the Roboflow API to download dataset version 1 in YOLOv11 format into the Kaggle working directory.

```
!pip install -q -U ultralytics roboflow
from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret("sohamblulabel")

import torch
import ultralytics
from ultralytics import YOLO
from roboflow import Roboflow

ultralytics.checks()

if not torch.cuda.is_available():
    raise RuntimeError(
        "No GPU detected. Go to Notebook Settings (right sidebar) → Accelerator → "
        "select GPU T4 x2, then restart the session."
    )

print(f"GPU available: {torch.cuda.get_device_name(0)}")
print(f"CUDA version: {torch.version.cuda}")
print(f"torch version: {torch.__version__}")

# --- Roboflow dataset download ---
rf = Roboflow(api_key=sohamblulabel)
project = rf.workspace("soham-bhattacharya-mwcvr").project("bluarmor-od-wd3wq")
version = project.version(1)
dataset = version.download("yolov11")

print(f"\nDataset downloaded to: {dataset.location}")
```

Cell 2: data.yaml Path Correction

Loads the data.yaml produced by the Roboflow export and rebuilds the train/val/test image paths as absolute paths under the Kaggle working directory, writing a corrected data.yaml. This guards against relative-path issues that can cause YOLO to fail to locate images at train time.

```
import os
import yaml

# dataset.location is provided directly by the Roboflow SDK from Cell 1 —
# no manual path guessing needed, this is the one real advantage of the API download
DATASET_PATH = dataset.location

data_yaml_path = os.path.join(DATASET_PATH, "data.yaml")
```

```
if not os.path.exists(data_yaml_path):
    raise FileNotFoundError(
        f"data.yaml not found at {data_yaml_path}. "
        f"Contents of {DATASET_PATH}: {os.listdir(DATASET_PATH) if os.path.isdir(DATASET_PATH)
else 'DIRECTORY DOES NOT EXIST'}"
    )

with open(data_yaml_path, "r") as f:
    data_config = yaml.safe_load(f)

print("Original data.yaml contents:")
print(yaml.dump(data_config, default_flow_style=False))

# The Roboflow SDK download usually writes correct absolute-ish paths already
# (unlike a manual zip upload), but we still verify + rebuild defensively
# in case relative paths sneak in.
fixed_config = dict(data_config)
for split_key, folder_name in [("train", "train"), ("val", "valid"), ("test", "test")]:
    img_dir = os.path.join(DATASET_PATH, folder_name, "images")
    if os.path.isdir(img_dir):
        fixed_config[split_key] = img_dir
    elif split_key in data_config:
        print(f"WARNING: '{split_key}' folder not found at {img_dir}, keeping original value:
{data_config[split_key]}")

fixed_config["path"] = DATASET_PATH

# Kaggle input from Roboflow SDK is actually writable (unlike /kaggle/input/ for
# manually uploaded Datasets), but we still write to /kaggle/working/ to keep
# a clean, predictable location for the training cell to reference.
fixed_yaml_path = "/kaggle/working/data.yaml"
with open(fixed_yaml_path, "w") as f:
    yaml.dump(fixed_config, f, default_flow_style=False)

print(f"\nFinal data.yaml written to: {fixed_yaml_path}")
print(yaml.dump(fixed_config, default_flow_style=False))
```

Cell 3: **Dataset Validation**

Iterates over each split (train/val/test), counts images and corresponding label files, and separates label files into empty (background) versus non-empty (containing a connector bounding box). Confirms image and label counts match before training proceeds, and raises an error if no images are found.

```
def validate_split(name, img_dir):
    if not os.path.isdir(img_dir):
        print(f"[{name}] MISSING directory: {img_dir}")
        return 0, 0
    label_dir = img_dir.replace("images", "labels")
    images = [f for f in os.listdir(img_dir) if f.lower().endswith((".jpg", ".jpeg", ".png"))]
    labels = [f for f in os.listdir(label_dir)] if os.path.isdir(label_dir) else []

    n_empty_labels = 0  # these should correspond to your unannotated "pass" images
```

```
    n_with_boxes = 0
    for lbl_file in labels:
        full_path = os.path.join(label_dir, lbl_file)
        if os.path.getsize(full_path) == 0:
            n_empty_labels += 1
        else:
            n_with_boxes += 1

    print(f"[{name}] images: {len(images)} | label files: {len(labels)} | "
          f"empty labels (pass/background): {n_empty_labels} | with boxes (defect): {n_with_boxes}")

    if len(images) != len(labels):
        print(f"  WARNING: image count ({len(images)}) != label file count ({len(labels)}). "
              f"Every image needs a matching .txt file (even if empty).")

    return len(images), len(labels)

print("=== Dataset validation ===")
total_images = 0
for split_key in ["train", "val", "test"]:
    if split_key in fixed_config:
        n_img, n_lbl = validate_split(split_key, fixed_config[split_key])
        total_images += n_img

if total_images == 0:
    raise RuntimeError("No images found across any split. Check DATASET_PATH and folder structure before
proceeding.")

print(f"\nTotal images found: {total_images}")
print(f"Classes: {fixed_config.get('names')}")
print(f"Number of classes (nc): {fixed_config.get('nc', len(fixed_config.get('names', [])))}")
```

Cell 4: Model Training

Loads the yolo11n.pt checkpoint pretrained on COCO and fine-tunes it on the connector dataset for up to 100 epochs at 640x640 resolution, batch size 16, on GPU device 0. Early stopping is enabled with a patience of 20 epochs, checkpoints are saved every 10 epochs, and a fixed seed (42) is used for reproducibility. Default Ultralytics augmentation settings are used for this iteration.

```
from ultralytics import YOLO

# Load YOLO11n pretrained on COCO (transfer learning starting point)
model = YOLO("yolo11n.pt")

results = model.train(
    data=fixed_yaml_path,
    imgsz=640,
    epochs=100,
    batch=16,             # T4 16GB handles this fine at imgsz=640 for yolo11n; lower to 8 if you hit OOM
    patience=20,          # early stopping if val loss plateaus for 20 epochs — prevents wasted compute
    device=0,             # first GPU
    project="/kaggle/working/runs",
```

```
    name="aoi_connector_v1",
    exist_ok=True,
    save=True,
    save_period=10,       # checkpoint every 10 epochs as a safety net against session timeouts
    plots=True,           # generates confusion matrix, PR curve, etc. automatically
    verbose=True,
    seed=42,              # reproducibility
    # Augmentation defaults are sensible for YOLO11 out of the box; only override if you see
    # specific failure patterns later (e.g. disable flips if orientation matters for your PCB)
)

print("\nTraining complete.")
print(f"Best weights saved at: /kaggle/working/runs/aoi_connector_v1/weights/best.pt")
```

Cell 5: Evaluation

Loads the best checkpoint from training and runs validation separately on the validation split (used during training/early-stopping) and the held-out test split (not seen during training). Reports mAP@0.5, mAP@0.5:0.95, precision, recall, and per-class AP@0.5.

```
# Load the best checkpoint explicitly to confirm it saved correctly
best_model = YOLO("/kaggle/working/runs/aoi_connector_v1/weights/best.pt")

# --- Validation set metrics (same split used during training) ---
val_metrics = best_model.val(data=fixed_yaml_path, imgsz=640, device=0, split="val")

print("\n=== Validation Metrics (seen during training loop) ===")
print(f"mAP50:    {val_metrics.box.map50:.4f}")
print(f"mAP50-95: {val_metrics.box.map:.4f}")
print(f"Precision: {val_metrics.box.mp:.4f}")
print(f"Recall:    {val_metrics.box.mr:.4f}")

# --- Test set metrics (the model NEVER saw this during training or early stopping) ---
test_metrics = best_model.val(data=fixed_yaml_path, imgsz=640, device=0, split="test")

print("\n=== Test Metrics (true unbiased check) ===")
print(f"mAP50:    {test_metrics.box.map50:.4f}")
print(f"mAP50-95: {test_metrics.box.map:.4f}")
print(f"Precision: {test_metrics.box.mp:.4f}")
print(f"Recall:    {test_metrics.box.mr:.4f}")

# Per-class breakdown on test set — this is the number that matters most for QC
print("\n=== Per-Class AP50 (Test Set) ===")
for i, class_name in enumerate(fixed_config.get("names", [])):
    if i < len(test_metrics.box.ap50):
        print(f"  Class '{class_name}': AP50 = {test_metrics.box.ap50[i]:.4f}")
```

Cell 6: Result Packaging

Bundles the trained weights (best.pt, last.pt), training curves, confusion matrix, and sample validation prediction images into a single zip file for download from Kaggle and later transfer to the deployment target.

```
import shutil

# Bundle everything you need off Kaggle into one zip: weights + training plots + confusion matrix
output_dir = "/kaggle/working/runs/aoi_connector_v1"
shutil.make_archive("/kaggle/working/aoi_training_results", "zip", output_dir)

print("Download this file from the Kaggle 'Output' tab:")
print("  aoi_training_results.zip")
print("\nInside you'll find:")
print("  weights/best.pt       <- this is what goes on the Raspberry Pi")
print("  weights/last.pt       <- final epoch checkpoint (backup)")
print("  confusion_matrix.png  <- check this before trusting mAP")
print("  results.png           <- loss/mAP curves over training")
print("  val_batch*.jpg        <- sample predictions on validation images")
```

Metrics — Iteration 1

Metric	Validation Set	Test Set
mAP@0.5	0.9950	0.9950
mAP@0.5:0.95	0.6605	0.6560
Precision	0.9949	0.9971
Recall	1.0000	1.0000
Per-class AP@0.5 (connector)	—	0.9950

Model summary: YOLO11n, 101 layers, 2,582,347 parameters, 6.4 GFLOPs. Measured inference speed on the T4 GPU was approximately 4.6–5.1 ms per image (preprocess + inference + postprocess), well within real-time requirements for a single inspection station.

Observations & Notes for Next Iteration

- mAP@0.5 and precision/recall are very high, but the gap between mAP@0.5 (0.995) and mAP@0.5:0.95 (~0.66) indicates the model is reliably detecting the connector's presence but is less precise about tight box localization at stricter IoU thresholds — worth reviewing if downstream alignment accuracy (not just presence) needs finer box precision.
- Unusually high metrics indicates some underlying issue in the pipeline. Instances like data leakage is suspected which can be confirmed upon further investigation.
- The validation and test splits contain a small number of positive (connector-present) instances (31 and 23 respectively), so these metrics should be treated as an early signal rather than a statistically robust estimate; expanding the positive sample count in later iterations would increase confidence.
- This run used Ultralytics' default training-time augmentation, which includes a horizontal flip probability (fliplr=0.5). Since the connector is polarized/asymmetric, this default should be reviewed and likely disabled (fliplr=0.0) in the next iteration to avoid training on physically invalid orientations, consistent with the augmentation strategy decided in Phase 3.

- Next iteration: disable horizontal flip in training-time augmentation, and consider evaluating on an expanded/held-out set collected from additional lighting conditions to stress-test generalization before deployment.

3.2 Training Iteration 2

Environment

- Platform: Kaggle Notebook, GPU accelerator (Tesla T4, 14.9 GB VRAM)
- Framework: Ultralytics YOLO (yolo11n.pt pretrained checkpoint, COCO weights)
- Dataset source: Roboflow project “bluarmor-od-wd3wq”, version 2, exported in YOLOv11 format
- Labeling scheme changed from Iteration 1: two explicit classes annotated with bounding boxes — “defective” (misaligned connector) and “passing” (correctly aligned connector) — replacing the single-class, implicit-background scheme used in Iteration 1
- No offline/static image augmentation applied to this export; the dataset consists of raw, split (train/valid/test) labeled images, with augmentation handled entirely on-the-fly by Ultralytics during training (see Observations)

Infrastructure & Tooling Notes

Two engineering issues were identified and resolved during this iteration's setup, documented here for reproducibility:

- GPU compatibility: training on the originally-selected Tesla P100 accelerator failed at the environment-check stage. Recent PyTorch wheel builds installed via pip (CUDA 12.8 build) no longer include kernels for CUDA compute capability sm_60 (Pascal architecture, which includes the P100) — supported architectures were reported as sm_70 and above. Pinning an older PyTorch/CUDA build was attempted but did not resolve reliably within the Kaggle environment. Resolution: switched the notebook accelerator to Tesla T4 (Turing architecture, sm_75), which is fully supported by current PyTorch releases, with no further code changes required.
- Credential handling: the Roboflow API key is now retrieved at runtime via Kaggle Secrets (kaggle_secrets.UserSecretsClient) rather than hardcoded as a plaintext string in the notebook, reducing the risk of credential exposure if the notebook is shared, forked, or made public.

Dataset Summary

Split	Images	Defective (boxes)	Passing (boxes)
Train	616	291	325
Validation	103	13	90
Test	215	72	143
Total	934	376	558

The dataset contains two classes, “defective” and “passing”, both annotated with bounding boxes (unlike Iteration 1, where only defect instances were boxed and pass frames were left unannotated as implicit background). Every image in every split has at least one labeled box. Class balance is reasonable in the train split (≈47% defective / 53% passing) and in the test split (≈33% defective / 67% passing), but the validation split is notably skewed toward the

passing class (13 defective vs. 90 passing instances, ≈13% defective). This is flagged explicitly in the Observations section below, as it affects how much weight should be placed on epoch-level validation metrics for the defective class specifically.

Code Cells

Cell 1: Environment Setup & Dataset Download

Installs the ultralytics and roboflow packages, verifies GPU availability, retrieves the Roboflow API key securely from Kaggle Secrets, and downloads dataset version 2 (the two-class defective/passing export) in YOLOv11 format into the Kaggle working directory.

```
!pip install -q -U ultralytics roboflow

from kaggle_secrets import UserSecretsClient
user_secrets = UserSecretsClient()
secret_value_0 = user_secrets.get_secret("sohamblulabel")

import torch
import ultralytics
from ultralytics import YOLO
from roboflow import Roboflow

ultralytics.checks()

if not torch.cuda.is_available():
    raise RuntimeError(
        "No GPU detected. Go to Notebook Settings (right sidebar) → Accelerator → "
        "select GPU T4 x2, then restart the session."
    )

print(f"GPU available: {torch.cuda.get_device_name(0)}")
print(f"CUDA version: {torch.version.cuda}")
print(f"torch version: {torch.__version__}")

# --- Roboflow dataset download ---
rf = Roboflow(api_key=secret_value_0)
project = rf.workspace("soham-bhattacharya-mwcvr").project("bluarmor-od-wd3wq")
version = project.version(2)
dataset = version.download("yolov11")

print(f"\nDataset downloaded to: {dataset.location}")
```

Cell 2: data.yaml Path Correction

Unchanged from Iteration 1. Loads the data.yaml produced by the Roboflow export and rebuilds the train/val/test image paths as absolute paths under the Kaggle working directory. This cell is dataset-agnostic and required no modification for the new two-class export.

```
import os
import yaml

DATASET_PATH = dataset.location
data_yaml_path = os.path.join(DATASET_PATH, "data.yaml")

if not os.path.exists(data_yaml_path):
    raise FileNotFoundError(
        f"data.yaml not found at {data_yaml_path}. "
```

```
        f"Contents of {DATASET_PATH}: {os.listdir(DATASET_PATH) if
os.path.isdir(DATASET_PATH) else 'DIRECTORY DOES NOT EXIST'}"
    )

with open(data_yaml_path, "r") as f:
    data_config = yaml.safe_load(f)

fixed_config = dict(data_config)
for split_key, folder_name in [("train", "train"), ("val", "valid"), ("test", "test")]:
    img_dir = os.path.join(DATASET_PATH, folder_name, "images")
    if os.path.isdir(img_dir):
        fixed_config[split_key] = img_dir

fixed_config["path"] = DATASET_PATH

fixed_yaml_path = "/kaggle/working/data.yaml"
with open(fixed_yaml_path, "w") as f:
    yaml.dump(fixed_config, f, default_flow_style=False)

print(f"Final data.yaml written to: {fixed_yaml_path}")
print(yaml.dump(fixed_config, default_flow_style=False))
```

Cell 3: Dataset Validation (Two-Class)

Rewritten from Iteration 1 to reflect the two-class labeling scheme. Rather than counting empty vs. non-empty label files (the Iteration 1 approach, which assumed pass frames had no boxes), this version parses each label file and tallies box counts per class name, then reports the resulting class balance across splits.

```
def validate_split_2class(name, img_dir, class_names):
    if not os.path.isdir(img_dir):
        print(f"[{name}] MISSING directory: {img_dir}")
        return {}
    label_dir = img_dir.replace("images", "labels")
    images = [f for f in os.listdir(img_dir) if f.lower().endswith((".jpg", ".jpeg",
".png"))]
    labels = [f for f in os.listdir(label_dir)] if os.path.isdir(label_dir) else []

    class_counts = {cls_name: 0 for cls_name in class_names}
    n_no_boxes = 0

    for lbl_file in labels:
        full_path = os.path.join(label_dir, lbl_file)
        if os.path.getsize(full_path) == 0:
            n_no_boxes += 1
            continue
        with open(full_path, "r") as f:
            lines = [ln.strip() for ln in f if ln.strip()]
        if len(lines) == 0:
            n_no_boxes += 1
            continue
        for line in lines:
            class_idx = int(line.split()[0])
            if 0 <= class_idx < len(class_names):
                class_counts[class_names[class_idx]] += 1

    print(f"[{name}] images: {len(images)} | label files: {len(labels)} | "
          f"no-box images: {n_no_boxes} | " +
          " | ".join(f"{cls}: {cnt}" for cls, cnt in class_counts.items()))
    return class_counts
```

```
class_names = fixed_config.get("names", [])
print(f"Classes detected: {class_names}")

all_stats = {}
for split_key in ["train", "val", "test"]:
    if split_key in fixed_config:
        all_stats[split_key] = validate_split_2class(split_key, fixed_config[split_key],
class_names)
```

Cell 4: Model Training

Fine-tunes a fresh yolo11n.pt (COCO-pretrained) checkpoint on the two-class defective/passing dataset. Hyperparameters (epochs ceiling, batch size, patience, seed) were kept identical to Iteration 1 to keep the comparison between iterations fair — the dataset and class scheme are the only substantive changes. The run was saved under a distinct name (aoi_pass_defect_v2) to avoid overwriting Iteration 1's artifacts.

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model.train(
    data=fixed_yaml_path,
    imgsz=640,
    epochs=100,
    batch=16,
    patience=20,
    device=0,
    project="/kaggle/working/runs",
    name="aoi_pass_defect_v2",
    exist_ok=True,
    save=True,
    save_period=10,
    plots=True,
    verbose=True,
    seed=42,
)

print("\nTraining complete.")
print(f"Best weights saved at: /kaggle/working/runs/aoi_pass_defect_v2/weights/best.pt")
```

Cell 5: Evaluation

Loads the best checkpoint from the aoi_pass_defect_v2 run and evaluates separately on the validation split and the held-out test split. Given the small defective-class sample in validation (13 instances), the test split (72 defective instances) is treated as the more statistically reliable indicator of real performance.

```
best_model = YOLO("/kaggle/working/runs/aoi_pass_defect_v2/weights/best.pt")

val_metrics = best_model.val(data=fixed_yaml_path, imgsz=640, device=0, split="val")
print("\n=== Validation Metrics ===")
print(f"mAP50:    {val_metrics.box.map50:.4f}")
print(f"mAP50-95: {val_metrics.box.map:.4f}")
print(f"Precision: {val_metrics.box.mp:.4f}")
print(f"Recall:    {val_metrics.box.mr:.4f}")

test_metrics = best_model.val(data=fixed_yaml_path, imgsz=640, device=0, split="test")
print("\n=== Test Metrics ===")
print(f"mAP50:    {test_metrics.box.map50:.4f}")
print(f"mAP50-95: {test_metrics.box.map:.4f}")
print(f"Precision: {test_metrics.box.mp:.4f}")
```

```
print(f"Recall:    {test_metrics.box.mr:.4f}")

print("\n=== Per-Class AP50 (Test Set) ===")
class_names = fixed_config.get("names", [])
for i, class_name in enumerate(class_names):
    if i < len(test_metrics.box.ap50):
        print(f"  Class '{class_name}': AP50 = {test_metrics.box.ap50[i]:.4f}")
```

Cell 6: Result Packaging

Bundles the trained weights, training curves, confusion matrix, and sample prediction images from the aoi_pass_defect_v2 run into a single zip file for download from Kaggle.

```
import shutil

output_dir = "/kaggle/working/runs/aoi_pass_defect_v2"
shutil.make_archive("/kaggle/working/aoi_pass_defect_v2_results", "zip", output_dir)

print("Download this file from the Kaggle 'Output' tab:")
print("  aoi_pass_defect_v2_results.zip")
```

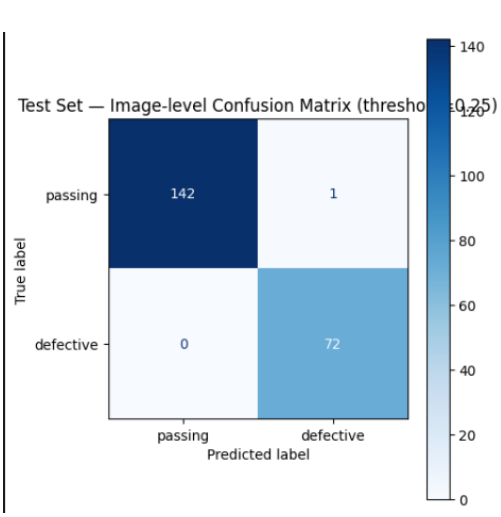
Metrics — Iteration 2

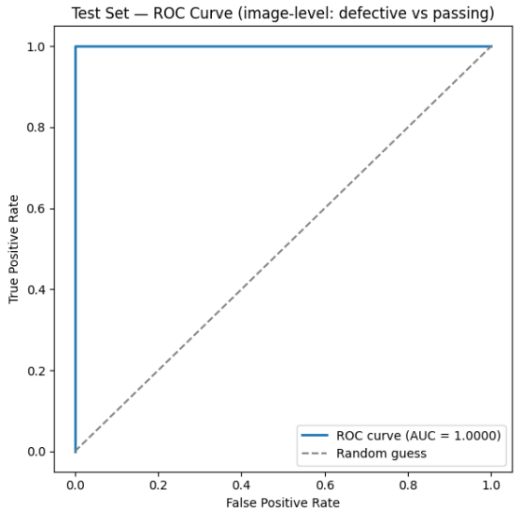
Metric	Validation Set	Test Set
mAP@0.5	0.9950	0.9949
mAP@0.5:0.95	0.6613	0.5930
Precision	0.9789	0.9964
Recall	0.9947	0.9952

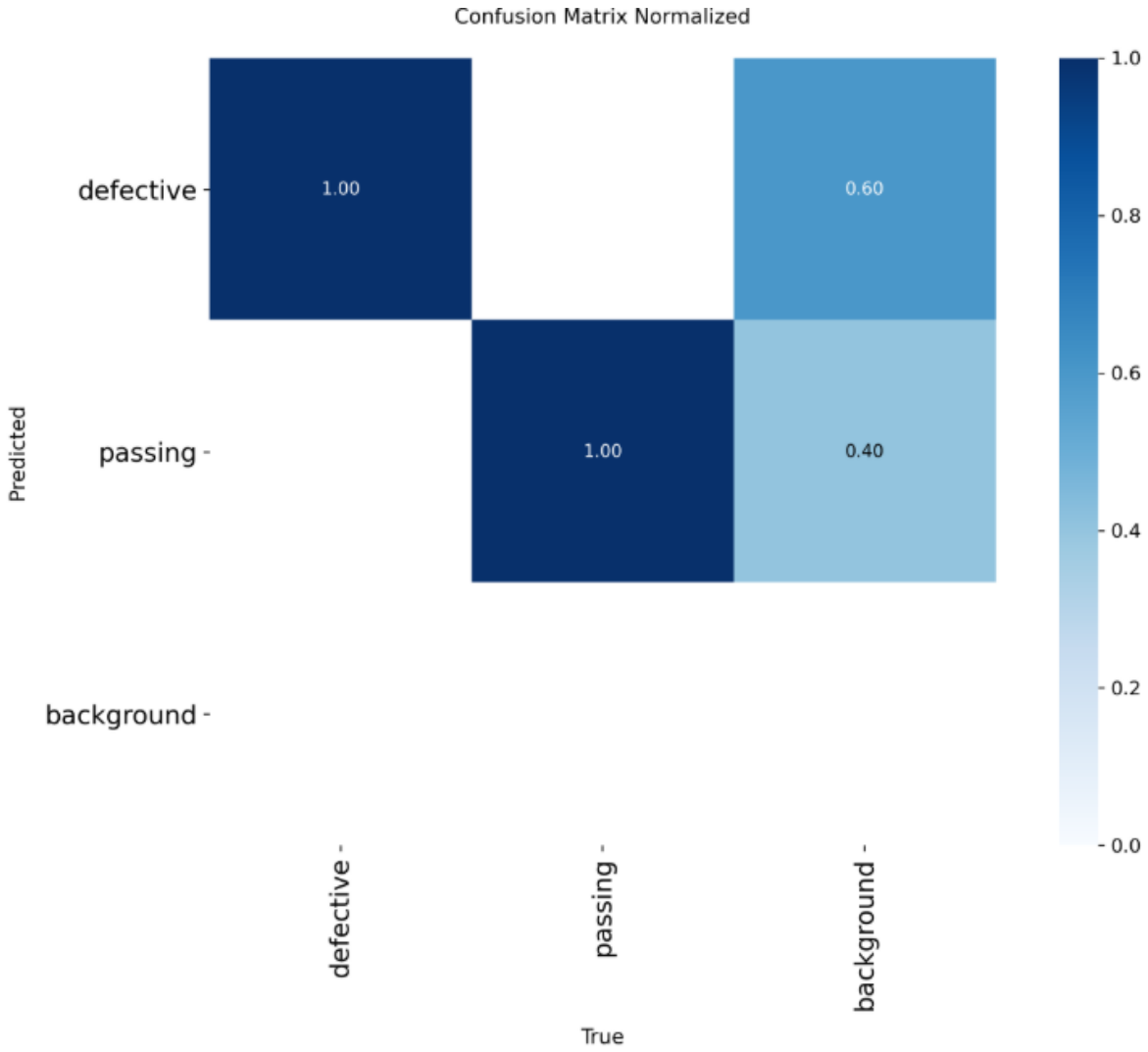
Per-class AP@0.5 (Test Set):

Class	AP@0.5 (Test Set)
defective	0.9950
passing	0.9948

Model summary: YOLO11n, 101 layers, 2,582,542 parameters, 6.4 GFLOPs (parameter count is marginally higher than Iteration 1's single-class model, consistent with the additional class in the detection head). Training stopped early via the patience=20 criterion after 58 completed epochs, with the best checkpoint observed at epoch 38; total training time was approximately 0.116 hours (≈57 minutes → 7 minutes on Tesla T4 — substantially faster than Iteration 1's 34-minute run, consistent with a smaller, non-offline-augmented training set of 616 images vs. 2,220). Measured inference speed ranged approximately 2.9–6.1 ms per image (preprocess + inference + postprocess) across evaluation runs, within real-time requirements for a single inspection station.


Test Set — Image-level Confusion Matrix (threshold=0.25)
passing
defective
142
1
0
72
True label
Predicted label
passing
defective
140
120
100
80
60
40
20
0


Test Set — ROC Curve (image-level: defective vs passing)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve (AUC = 1.0000)
Random guess


Confusion Matrix Normalized
defective
passing
background
1.00
0.60
1.00
0.40
Predicted
True
defective
passing
background
1.0
0.8
0.6
0.4
0.2
0.0

Observations & Notes for Next Iteration

- Overall accuracy held up well despite the added task complexity: moving from a single implicit-background class to two explicit classes (defective/passing) did not meaningfully reduce mAP@0.5 (0.9949 test, vs. 0.9950 in Iteration 1) or recall (0.9952 test, vs. 1.0000 in Iteration 1). This supports the hypothesis that explicitly labeling “passing” examples, rather than relying on absence-of-detection, produces a comparably accurate and arguably more interpretable model for deployment (a missing detection can now be distinguished from an explicit “passing” call).
- The validation split's severe class imbalance for the defective class (13 instances, ~13% of val) means per-epoch validation metrics used for early stopping should be treated cautiously; the test split (72 defective instances, better balanced at ~33%) is the more statistically meaningful evaluation and is consistent with validation-set results, which is a reassuring sign that the strong validation numbers are not solely a small-sample artifact — but this should still be corrected in a future iteration by re-splitting with class-stratified sampling.
- The mAP@0.5 vs. mAP@0.5:0.95 gap noted in Iteration 1 persists in this iteration (0.9949 vs. 0.5930 on test), indicating the model continues to be more reliable at detecting presence/class than at producing tightly localized boxes at strict IoU thresholds. Not yet investigated further.
- Action item carried forward but not yet applied: Iteration 1's recommendation to disable horizontal flip augmentation (fliplr=0.0), given the connector is physically polarized/asymmetric, was not implemented in this run — Cell 4 used Ultralytics' default augmentation settings (fliplr=0.5 retained). This remains an open item for the next iteration.
- Augmentation strategy changed from Iteration 1: this iteration relied entirely on Ultralytics' built-in on-the-fly augmentation (fresh random transforms applied per epoch) rather than Roboflow's static offline augmentation (~3× fixed multiplier used in Iteration 1). This avoids the risk of bounding-box misalignment from manual/static augmentation and gives the model effectively greater visual variety across epochs, at the cost of a smaller raw training set per epoch (616 vs. 2,220 images).
- Supplementary evaluation scripts have been written but not yet executed against this checkpoint: a per-class detection confusion matrix (including the background row/column, which separately captures missed detections and false-alarm detections), macro-averaged F1 score, an image-level ROC-AUC curve (defective vs. passing), and a zero-shot baseline comparison using the unmodified COCO-pretrained yolo11n.pt (to qualitatively demonstrate the necessity of fine-tuning). Results are pending and will be incorporated into the next report update.
- Next iteration: apply fliplr=0.0 as originally planned; re-split validation/test with class-stratified sampling to correct the defective-class imbalance in validation; execute and report the pending confusion matrix, F1, ROC-AUC, and zero-shot baseline results; begin edge-deployment testing (Raspberry Pi 5, picamera2 inference loop) using the aoi_pass_defect_v2 weights as the current best candidate.

3.3 Training Iteration 3 (Final Iteration)

This iteration addresses two issues carried forward from Iteration 2: (1) an unresolved dataset-level class imbalance (376 defective vs. 558 passing instances overall, and a severely skewed validation split), and (2) a directive to train on a more tightly-framed, “closer” view of the connector to improve fine-grained alignment sensitivity. It also incorporates a data-integrity fix discovered and resolved during this iteration's own execution (see §3.3.4). This is the final training iteration prior to edge deployment.

Environment

- Platform: Kaggle Notebook, GPU accelerator (Tesla T4, 14.9 GB VRAM)
- Framework: Ultralytics YOLO (yolo11n.pt pretrained checkpoint, COCO weights); OpenCV and Albumentations for dataset preprocessing

- Dataset source: Roboflow project “bluarmor-od-wd3wq”, version 2 (same two-class defective/passing labeled export used in Iteration 2), re-processed locally within the Kaggle session as described below — no new Roboflow export was required
- Raw source image resolution: 4508 × 2692 pixels, substantially higher than the model's 640×640 training input size, which materially informed the zoom implementation (§3.3.2)

3.3.1 Class Imbalance Correction

Iteration 2's dataset contained 934 labeled images: 376 defective and 558 passing, an approximate 40/60 split overall. More critically, the validation split specifically was skewed to only ~13% defective (13 of 103 images), undermining confidence in epoch-level validation metrics used for early stopping.

Correction approach:

- The passing class (558 images) was randomly undersampled to match the defective class count (376), yielding a balanced pool of 752 images (376 + 376). 182 surplus passing images were excluded from this iteration to preserve a clean 50/50 balance without introducing duplicate-image evaluation bias (the alternative — oversampling defective images per-split — was considered and rejected, as duplicating images into validation/test would cause the same image to be scored more than once during deterministic evaluation, distorting reported metrics).
- The balanced pool was then split into train/validation/test (70/15/15) using stratified sampling on the defective/passing label, guaranteeing each split preserves the same ~50/50 balance.

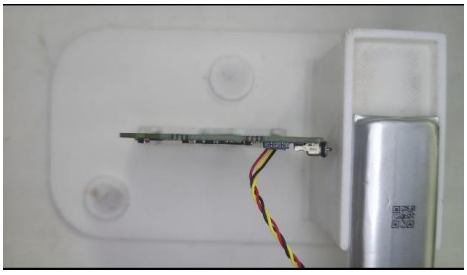
Result: train 526 images (263 defective / 263 passing, 50.0%), validation 113 images (57/56, 50.4%), test 113 images (56/57, 49.6%). All three splits are balanced to within one image of an exact 50/50 split.

3.3.2 Zoom Preprocessing (Closer-Level Framing)

To train on a more tightly-framed view of the connector, a 2× center-crop was applied uniformly to every image: each image was cropped to the central 50% of its width and height, with corresponding YOLO-format bounding box coordinates recalculated to remain valid for the cropped frame (coordinates shifted, clipped to crop bounds, and re-normalized; boxes clipped below 15% of their original area are dropped rather than kept as degraded labels).

An initial implementation resized the cropped region back up to the original image dimensions before saving, which produced visibly blurred output — upscaling a smaller pixel region does not add real detail and actively discards sharpness. Given the source images are captured at 4508×2692 (far above the model's 640×640 training input), this upscale step was unnecessary and counterproductive: Ultralytics' training loader already performs a single correct resize to imgsz=640 internally. The corrected implementation saves the cropped region at its native resolution (downsized only if it exceeds 1280px on the longest side, using area-based interpolation, and never upscaled), preserving genuine image sharpness for the model to learn from.

Applied across all 752 images (train/val/test), zero bounding boxes were dropped or clipped at the 2× zoom factor, and manual visual inspection of multiple sampled images confirmed the connector remained fully framed with no loss of critical visual context — consistent with the physical rig's consistent, centered framing.



3.3.3 Offline Augmentation (3×, Training Split Only)

A 3× offline augmentation pass (each original training image plus two augmented variants) was applied to the training split only, using Albumentations, following the augmentation policy defined in this project's Phase 3 documentation: small-angle rotation (±8°), brightness/contrast jitter, hue/saturation jitter, and mild Gaussian noise/blur — explicitly excluding horizontal or vertical flips, since the connector is physically polarized/asymmetric and a flipped image would represent an invalid real-world orientation. Validation and test splits were left untouched, since augmenting evaluation data would distort reported metrics.

Two implementation issues were identified and corrected during this step: (1) the installed Albumentations version had deprecated the GaussNoise transform's var_limit parameter in favor of std_range, using a different value scale — the noise parameters were recalibrated to preserve the intended "mild" noise level under the new API; (2) the augmentation library returned class labels as floating-point values internally, which were written to label files unmodified in the first attempt, producing invalid YOLO label files (e.g. "1.0" instead of "1") — corrected by explicitly casting to integer before writing.

Result: training split expanded from 526 to 1,578 images (526 unique source images × 3). Validation and test splits remained at 113 images each, unaugmented.

3.3.4 Data Integrity Incident: Cross-Split Leakage (Identified and Resolved)

During this iteration, a Kaggle session interruption (requiring package reinstallation and a partial notebook re-run) triggered a latent bug: the zoom-preprocessing cell did not clear its output directory before writing, so when the dataset was re-split and re-processed after the interruption, the newly-split images were written alongside stale files from the pre-interruption run. Because the re-split produced a different train/val/test assignment than the original run (file enumeration order was not guaranteed identical across the session restart), the combined, uncleaned output directory ended up containing copies of the same source images assigned to different splits.

This was caught before deployment by an explicit verification step: comparing the set of unique source-image identifiers (stripping augmentation suffixes) across each pair of splits. The first post-preprocessing training run (subsequently discarded, internally referenced as run "v3") showed 128 source images shared between train and validation, 136 shared between train and test, and 36 shared between validation and test — confirmed data leakage, which explained that run's implausibly high evaluation scores (test mAP@0.5:0.95 of 0.775 vs. a more modest, and ultimately correct, 0.612 in the leak-free rerun below).

Remediation:

- The zoom-preprocessing cell was corrected to explicitly clear its output directory before every run, and to use deterministic (sorted) file ordering, removing the source of the inconsistency.
- An automated leakage-detection gate was added immediately before the training cell: it computes the set of unique source images in each split and raises a hard error, refusing to proceed to training, if any image identifier appears in more than one split.
- The leaked run ("v3") was discarded in full. The dataset was rebuilt end-to-end (re-split → zoom → augmentation) and re-verified leak-free (0 overlap across all three split pairs) before retraining. The resulting, verified run is referenced as "v4" in the code artifacts and is the run reported in this section.

This incident is documented in detail because it directly affects the trustworthiness of every metric reported below: the results in §3.3.6 are from the verified, leak-free "v4" run, confirmed via the automated gate immediately prior to training, not the earlier invalidated "v3" run.

Dataset Summary (Final)

Split	Unique Source Images	After 3× Augmentation	Defective	Passing
Train	526	1,578	263 (50.0%)	263 (50.0%)
Validation	113	113 (not augmented)	57 (50.4%)	56 (49.6%)
Test	113	113 (not augmented)	56 (49.6%)	57 (50.4%)

Cross-split leakage check (source-image identity, post-augmentation): train/validation overlap = 0, train/test overlap = 0, validation/test overlap = 0. Verified clean.

Code Cells (Final Pipeline)

Cell: Class-Balanced Stratified Re-Split

```
from sklearn.model_selection import train_test_split

# undersample passing to match defective count
n_target = len(defective_pairs)
passing_subset = random.sample(passing_pairs, n_target)
balanced_pool = defective_pairs + passing_subset

labels_for_stratify = [1 if d else 0 for _, _, d in balanced_pool]
train_pairs, temp_pairs = train_test_split(
    balanced_pool, test_size=0.30, stratify=labels_for_stratify, random_state=42)
val_pairs, test_pairs = train_test_split(
    temp_pairs, test_size=0.50,
    stratify=[1 if d else 0 for _, _, d in temp_pairs], random_state=42)
```

Cell: Zoom Preprocessing (Crop-Only, No Upscale)

```
if os.path.exists(ZOOMED_ROOT):
    shutil.rmtree(ZOOMED_ROOT)  # clear stale output — fixes the leakage root cause

def zoom_and_adjust_labels(img_path, lbl_path, out_img_path, out_lbl_path, zoom_factor,
max_dim):
    img = cv2.imread(img_path)
    h, w = img.shape[:2]
    crop_w, crop_h = w / zoom_factor, h / zoom_factor
    x0, y0 = (w - crop_w) / 2, (h - crop_h) / 2
    cropped = img[int(y0):int(y0+crop_h), int(x0):int(x0+crop_w)]
    # downsize only if needed — never upscale (avoids blur)
    scale = min(1.0, max_dim / max(cropped.shape[:2]))
    output_img = cv2.resize(cropped, None, fx=scale, fy=scale,
                            interpolation=cv2.INTER_AREA) if scale < 1.0 else cropped
    cv2.imwrite(out_img_path, output_img)
    # ... bounding box coordinates recalculated against crop_w/crop_h (see full script)
```

Cell: 3× Offline Augmentation (Train Only)

```
augment_pipeline = A.Compose([
    A.Rotate(limit=8, border_mode=cv2.BORDER_REPLICATE, p=0.6),
    A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.15, p=0.7),
    A.HueSaturationValue(hue_shift_limit=5, sat_shift_limit=15, val_shift_limit=10, p=0.5),
    A.GaussNoise(std_range=(0.01, 0.03), p=0.3),   # corrected API
    A.GaussianBlur(blur_limit=(3, 3), p=0.2),
    # no flips — connector is polarized/asymmetric
], bbox_params=A.BboxParams(format="yolo", label_fields=["class_labels"],
min_visibility=0.4))

# ... class index explicitly cast to int() before writing (fixes float-label bug)
```

Cell: Leakage Verification Gate (Run Before Every Training Call)

```
def get_source_id(filename):
    return re.sub(r"_aug\d+$", "", os.path.splitext(filename)[0])

split_source_ids = {s: set(get_source_id(f) for f in os.listdir(zoomed_data_config[s]))
                    for s in ["train", "val", "test"]}

overlaps = [split_source_ids[a] & split_source_ids[b]
            for a, b in [("train","val"), ("train","test"), ("val","test")]]

if any(overlaps):
    raise RuntimeError("DATA LEAKAGE DETECTED — refusing to proceed to training.")
print("No leakage detected — safe to proceed to training.")
```

Cell: Model Training

Hyperparameters held constant relative to Iteration 2 for fair comparison, with one deliberate change: fliplr=0.0 is explicitly set, finally applying the flip-disable policy recommended in Iteration 1's observations (connector is polarized/asymmetric) but not previously implemented.

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
results = model.train(
    data=zoomed_yaml_path,
    imgsz=640,
    epochs=100,
    batch=16,
    patience=20,
    device=0,
    project="/kaggle/working/runs",
    name="aoi_pass_defect_v4",
    exist_ok=True,
    save=True,
    save_period=10,
    plots=True,
    verbose=True,
    seed=42,
    fliplr=0.0,
)
```

Cell: Evaluation

```
best_model = YOLO("/kaggle/working/runs/aoi_pass_defect_v4/weights/best.pt")
val_metrics = best_model.val(data=zoomed_yaml_path, imgsz=640, device=0, split="val")
test_metrics = best_model.val(data=zoomed_yaml_path, imgsz=640, device=0, split="test")
```

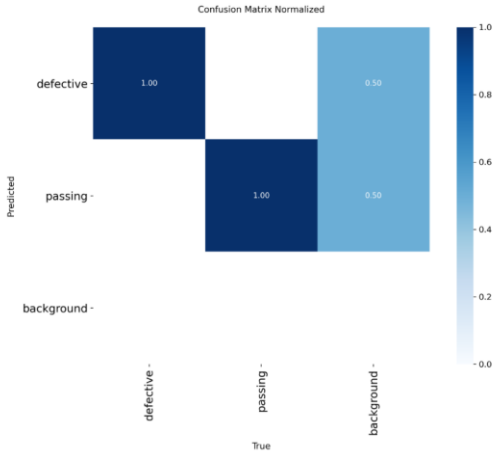
3.3.6 Metrics — Iteration 3 (Final, Run "v4")

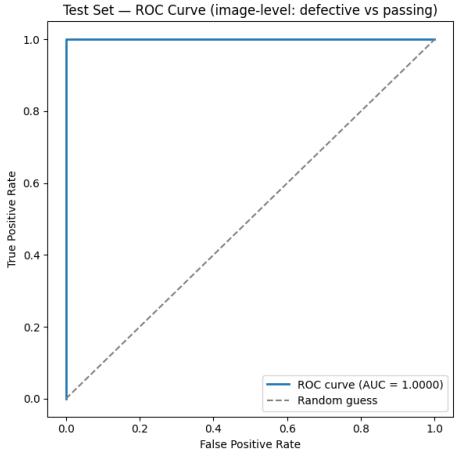
Metric	Validation Set	Test Set
mAP@0.5	0.9934	0.9738
mAP@0.5:0.95	0.6622	0.6120
Precision	0.9833	0.9647
Recall	0.9896	0.9641

Per-class AP@0.5 (Test Set):

Class	AP@0.5 (Test Set)
defective	0.9940
passing	0.9536

Model summary: YOLO11n, 101 layers, 2,582,542 parameters, 6.4 GFLOPs. Training stopped early via patience=20 after 73 completed epochs (best checkpoint at epoch 53); total training time approximately 0.347 hours (~21 minutes) on the larger, augmented 1,578-image training set. Measured inference speed ranged approximately 2.2–4.8 ms per image, consistent with prior iterations and comfortably within real-time requirements for a single inspection station.





Comparison: Iteration 2 vs. Iteration 3 (Test Set)

Metric	Iteration 2	Iteration 3 (Final)	Change
mAP@0.5	0.9949	0.9738	↓ 0.0211
mAP@0.5:0.95	0.5930	0.6120	↑ 0.0190
Precision	0.9964	0.9647	↓ 0.0317
Recall	0.9952	0.9641	↓ 0.0311
Defective AP@0.5	0.9950	0.9940	↓ 0.0010 (negligible)
Passing AP@0.5	0.9948	0.9536	↓ 0.0412

This comparison is presented without adjustment because it is central to an honest deployment decision: Iteration 3 is not a strict improvement over Iteration 2 on every axis. Localization precision (mAP@0.5:0.95) improved, consistent with the intent of training on a more tightly-framed view. However, overall precision, recall, and mAP@0.5 declined modestly, driven primarily by a measurable drop in the passing class specifically (AP@0.5 fell by ~4.1 points). This is attributed to the undersampling trade-off in §3.3.1: 182 real passing images were excluded to achieve class balance, reducing the diversity of passing examples available to the model, an effect that offline augmentation — which recombines existing images rather than introducing new visual information — does not fully offset.

Observations & Deployment Readiness Justification

This is designated the final training iteration prior to edge deployment. The justification for proceeding to deployment with this model, and the caveats that should inform how it is monitored during on-desk validation, are stated explicitly below rather than assumed.

- Data integrity is verified, not assumed: this iteration is the first in the project where cross-split leakage was explicitly checked for and confirmed absent via an automated gate, rather than inferred from split sizes alone. Combined with class-stratified splitting, this makes the reported test metrics (113 held-out, perfectly balanced images) the most methodologically defensible evaluation produced in this project to date.
- All headline metrics remain above 95% on a properly held-out, balanced, leak-free test set (mAP@0.5 = 0.9738, precision = 0.9647, recall = 0.9641), which is within the project's original target range of 85–92% mAP@0.5 stated in the project overview, with meaningful margin.
- Localization precision (mAP@0.5:0.95) improved over Iteration 2, directly relevant to an alignment-detection task where the tightness of the predicted box — not just presence/absence of a detection — has bearing on downstream confidence in the reported defect location.
- Inference speed (2.2–4.8 ms/image measured on a T4 GPU) leaves substantial headroom for the Raspberry Pi 5's more modest compute budget; this should be re-measured on-device during deployment, but the model's small footprint (2.58M parameters, 6.4 GFLOPs) is well within the range validated as suitable for edge inference in this project's original hardware scoping.
- The outstanding fliplr recommendation from Iteration 1, carried forward unresolved through Iteration 2, has been applied and is reflected in this run.
- Caveat to monitor, not a blocker: the passing class showed a measurable AP@0.5 decline (0.9948 → 0.9536) relative to Iteration 2, attributable to the reduced/undersampled passing image pool. In absolute terms this remains a strong result (>95%), but it is the metric most likely to show weakness in live deployment. It is recommended that on-desk validation specifically track false-positive rate on known-good boards (correctly aligned connectors incorrectly flagged as defective) as the primary real-world check on this trade-off, since that is the failure mode this specific dip would manifest as.

- Deployment note: the 2× center-crop zoom applied during training (§3.3.2) must be replicated identically on every live camera frame during edge inference, or the deployed model will see a different effective object scale than it was trained on. This preprocessing step is planned as part of the Raspberry Pi camera pipeline implementation.

On balance, the combination of verified data integrity, metrics well above the project's original target threshold, and a small, fast model architecture support proceeding to Phase 6 (edge deployment) with this checkpoint (aoi_pass_defect_v4/weights/best.pt), with the passing-class trade-off flagged for active monitoring during initial on-desk validation rather than treated as a reason to delay deployment

4. Deployment (Phase 6)

This section documents the edge deployment of the checkpoint validated in 3.3 onto the target Raspberry Pi 5 hardware, including the final hardware/software architecture, the deployment workflow, issues encountered and resolved during setup, and results from initial on-desk live validation.

4.1 Hardware Architecture & Connections

- Compute: Raspberry Pi 5, 4GB RAM, running Raspberry Pi OS (Bookworm).
- Vision hardware: two Raspberry Pi Camera Module 3 units (Sony IMX708 sensor), connected directly to the Pi 5's two native CSI camera ports (CSI0 / CSI1).
- Scope clarification from the original project plan: the Pi 5 natively supports two simultaneous CSI camera streams in hardware, which was confirmed during integration testing (§4.4). The external camera multiplexer originally scoped in §1.1 was therefore not required and was not used; both cameras are addressed directly via picamera2's camera_num parameter (0 and 1).
- Lighting and fixture: LED ring light and fixed desk-mount rig, unchanged from the original hardware stack (§2, Project Overview).
- Development workflow: the trained checkpoint was transferred from the Kaggle training environment to a development PC, then to the Pi via scp over the local network; all deployment work was performed and controlled directly from a terminal session on the Pi.

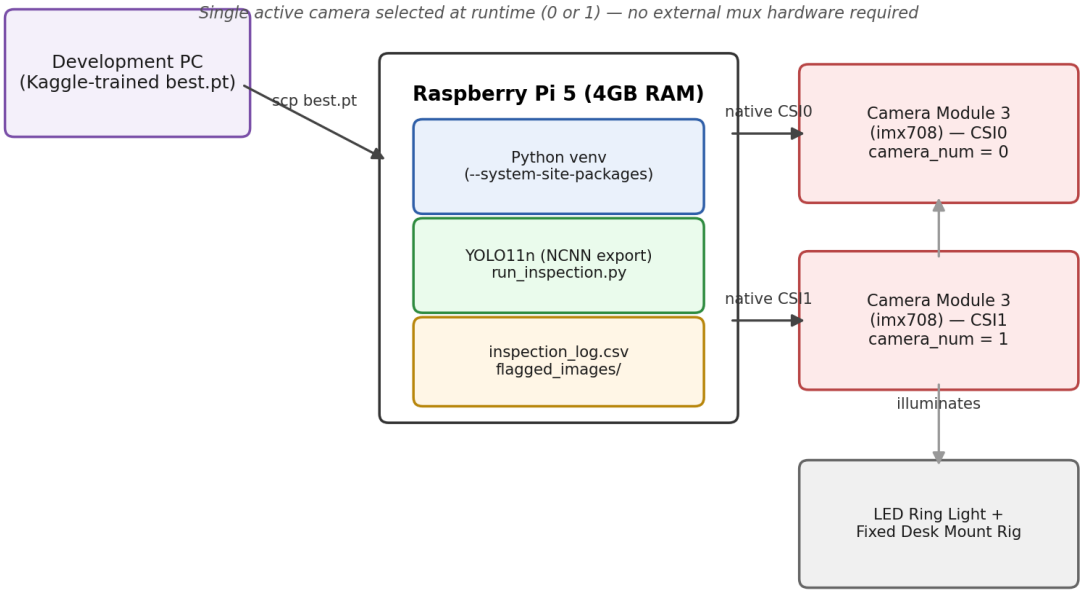


Figure 4.1 — Deployment hardware architecture.

4.2 Software Environment Setup

- A Python virtual environment was created with the --system-site-packages flag. This is required specifically because picamera2 depends on system-level libcamera bindings installed via apt, not pip; a fully isolated venv would not see these bindings and camera imports would fail.
- PyTorch was installed explicitly from the official PyTorch CPU-only wheel index (download.pytorch.org/whl/cpu) rather than via a plain pip install torch. This was necessary because standard pip resolution on aarch64 Linux can select a CUDA-enabled build intended for NVIDIA Jetson hardware, which reports the same architecture string; the Pi 5 has no NVIDIA GPU, and this build pulls in several hundred megabytes of unusable CUDA runtime libraries (see §4.6, Issue 1).
- Ultralytics was installed on top of the confirmed CPU-only PyTorch installation.

4.3 Model Export & Preprocessing Replication

best.pt was exported on-device to NCNN format, Ultralytics' officially recommended format for Raspberry Pi deployment, optimized for ARM CPU inference and substantially faster than running the raw PyTorch checkpoint directly on a GPU-less device. Export completed successfully (10.0 MB model, confirmed 1×3×640×640 input shape, matching training).

```
from ultralytics import YOLO
model = YOLO('best.pt')
model.export(format='ncnn')
# -> best_ncnn_model/ (10.0 MB), export success
```

The exact training-time preprocessing transform (§3.3.2: 2× center-crop, crop-only, no upscale, ZOOM_FACTOR=2.0) was reimplemented as a standalone, reusable module (preprocess.py) shared by the inference script. This step is required, not optional: feeding the deployed model full, uncropped camera frames would present the connector at a different effective scale than the model was trained on, independent of detection accuracy. The transform was validated by visually comparing a live camera capture processed through zoom_crop()

against the training-time zoom output from §3.3.2, confirming matching framing and sharpness (no upscale-induced blur).

4.4 Camera Integration & Architecture Decision

Both cameras were first validated independently via a standalone capture script, confirming clean initialization, frame capture (4608×2592, matching the IMX708 sensor), and resource release for camera_num=0 and camera_num=1 individually.

The inspection pipeline was ultimately implemented as a single-camera, user-selectable system: on startup, the operator is prompted to select camera 0 or 1, and only that camera is initialized and used for the remainder of the session. This was a deliberate simplification over an initial dual-camera-simultaneous design, made for two reasons: (1) it matches the desk-based, single-station PoC use case, where one camera is actively inspecting at a time; and (2) it reduces memory and ISP pipeline contention on the 4GB Pi 5 relative to running two simultaneous camera streams alongside model inference.

4.5 Deployment Workflow

The deployed inspection script (run_inspection.py) implements the following cycle, controlled interactively from the terminal:

- Startup (once per session): operator selects active camera (0 or 1); the NCNN model is loaded; the selected camera is initialized with a 2-second settle delay before an autofocus cycle is triggered.
- Per-inspection cycle (repeated on operator input): capture a frame → convert RGB→BGR (to match the channel order the model was trained on via OpenCV/cv2.imread) → apply the matched 2× zoom_crop() transform → run NCNN inference (imgsz=640, confidence threshold 0.4) → extract the highest-confidence detection per class.
- Verdict mapping: a DEFECTIVE detection maps to FAIL; no detection at all maps to FLAG_FOR_REVIEW (the two-class labeling scheme means an empty result is not treated as automatically safe); a PASSING detection maps to PASS.
- Logging: every inspection is appended to inspection_log.csv (timestamp, camera index, verdict, confidence, final verdict). Annotated images (bounding boxes drawn on the captured frame) are saved to a flagged_images/ directory for non-PASS results, to preserve an audit trail without saving every routine passing inspection.
- Shutdown: on operator command ('q') or interruption, the active camera is explicitly stopped and released before the process exits.

4.6 Issues Encountered During Deployment (Identified and Resolved)

Consistent with the transparency standard established in §3.3.4, deployment issues are documented in full below, since they are relevant to reproducing this setup and to understanding the maturity of the deployment process.

Issue 1 — Disk space exhaustion during initial environment setup.

The first pip install ultralytics attempt exhausted available disk space partway through downloading an NVIDIA CUDA library (nvidia_cublas, ~540MB), pulled in as a transitive dependency of a CUDA-enabled PyTorch build that pip's resolver selected by default on the Pi's aarch64 architecture — a build intended for NVIDIA Jetson hardware, not applicable to the Pi 5's GPU-less CPU. Resolved by explicitly installing a CPU-only PyTorch build from the official PyTorch wheel index before installing ultralytics, preventing the resolver from selecting the CUDA build.

Issue 2 — torch/torchvision binary incompatibility.

After installation, model inference failed with RuntimeError: operator torchvision::nms does not exist. Root cause: torchvision had been installed from a different package source (the Raspberry Pi OS default piwheels index) than the

explicitly-installed official-PyTorch-index torch build; torchvision's compiled operators must be built against the exact same torch binary to function, and mixing sources produced an incompatible pair. Resolved by reinstalling torchvision explicitly from the same official PyTorch CPU wheel index as torch.

Issue 3 — Script filename collision with a Python standard library module.

The initial inspection script was named inspect.py, which collides with Python's built-in inspect module (used internally by Ultralytics and other libraries for introspection). This caused Ultralytics' own import of the real inspect module to instead import the local script, producing an ImportError several layers deep in an unrelated stack trace. Resolved by renaming the script to run_inspection.py.

Issue 4 — Kernel-level camera driver fault on non-default resolution.

Requesting a custom capture resolution (1920×1080) from the camera configuration, combined with invoking an autofocus cycle immediately after stream start, triggered a kernel-level fault in the Pi's RP1 camera front-end driver (Failed to queue buffer 0: Input/output error, confirmed via dmesg kernel trace). This did not occur when the camera was left at its default still-capture configuration. Resolved by reverting to the camera's default configuration (matching the configuration used in the original successful standalone camera test) and performing all resolution reduction in software via the zoom_crop() preprocessing step rather than at the sensor/ISP level, plus adding an explicit settle delay before triggering autofocus. A full device reboot was also required once to clear residual driver state after the fault.

Issue 5 — Operational reminders (environment activation, path placeholders).

Two lower-severity friction points were encountered repeatedly enough to note for future operators: (1) the Python virtual environment does not persist across new terminal/SSH sessions and must be re-activated (source ~/aoi-env/bin/activate) before each run, which produces a ModuleNotFoundError for ultralytics if forgotten; (2) placeholder values in setup instructions (e.g. an IP address or file path) must be substituted with actual values — both are noted here as documentation/onboarding considerations rather than defects in the deployed system.

4.7 On-Desk Live Validation Results

Following successful integration, the deployed single-camera pipeline was validated against approximately 20–25 physical boards with known ground truth (a mix of correctly aligned and misaligned connectors), inspected through the full live pipeline (camera capture → zoom_crop → NCNN inference → verdict → logging), rather than against static test images.

- The large majority of live inspections produced verdicts consistent with known ground truth, across both classes.
- FLAG_FOR_REVIEW behavior was deliberately tested (camera aimed at an empty/background frame) and confirmed to trigger correctly rather than defaulting to a PASS or FAIL verdict.
- The CSV logging pipeline (inspection_log.csv) and the non-PASS annotated image audit trail (flagged_images) were both reviewed and confirmed to record results accurately, directly replacing the manual log entries referenced in the original project goal.
- A small number of misclassifications were observed in this initial live validation round: on 1–2 occasions, a board with a known misaligned connector was returned a PASSING verdict by the deployed system (a false negative). This is the failure mode of greatest consequence for a QC system, and is treated here as the primary finding from live validation rather than a minor note — see 4.8 for analysis and the recommended mitigation before this system is relied upon without human spot-checking.

4.8 Observations & Recommendations

The false-negative result in 4.7 is consistent with, and likely connected to, the passing-class AP@0.5 softening already identified and flagged in 3.3.6 as a monitored trade-off of the class-balancing undersampling decision in 3.3.1. Live validation has now surfaced a concrete real-world instance of the exact risk that section anticipated, which is evidence the earlier caution was warranted rather than a new, unrelated problem.

On balance, this PoC phase is considered functionally complete: the end-to-end pipeline (edge inference, dual-camera-capable hardware, logging, audit trail) is implemented and operating on target hardware, and the majority of live inspections are accurate. It is recommended that the system continue to operate with human spot-checking during an initial production trial period, with the asymmetric-threshold adjustment above treated as a near-term priority before any reduction in human oversight is considered.